CY376: NETWORK MONITORING, SECURITY AND AUDITING

End-of-Semester Project Report

Blue Team — Auditing Stored Procedures and Database Objects

for Security Weaknesses

Student Name: Veronica Okyere

Index Number: FCM.41,018.206.23

Track: Blue Team

Course Code: CY376

Date: August 2026

GitHub Repository:

https://github.com/cy-vokyere3623-hash/CY376-Blue-Team-Stored-Procedure-Audit

Abstract

Databases remain a high-value target because they store customer records, credentials, and business-critical data. Weak stored procedures, excessive permissions, and missing audit logging can allow attackers to steal data or escalate privileges from inside the database engine. This Blue Team project audited Microsoft SQL Server stored procedures and related database objects in an isolated laboratory using SQL Server 2022 Express, SQL Server Management Studio, and the AdventureWorks2022 sample database.

Intentionally vulnerable stored procedures were created to simulate SQL injection through unsafe dynamic SQL, privilege escalation through EXECUTE AS OWNER, operating-system command execution via an xp_cmdshell wrapper, and excessive EXECUTE grants to the public role. Custom Transact-SQL audit scripts aligned with CIS Microsoft SQL Server Benchmarks, OWASP SQL Injection Prevention guidance, Microsoft SQL Server Audit documentation, and NetSPI detective-control practices were then applied.

Six findings were identified and severity-rated. Remediation removed or replaced unsafe procedures, revoked public execute rights, enabled SQL Server Audit, and disabled the remote access configuration option. Re-running the same audit scripts verified that the critical and high risks were eliminated and that detective controls were active. The project demonstrates a complete blue-team workflow: inventory, detection, documentation, remediation, and verification.

Table of Contents

6. Analysis and Recommendations

7. Conclusion

8. References

9. Appendices

1. Introduction

1.1 Background

Blue team work focuses on defending systems by identifying weaknesses, improving controls, and ensuring that suspicious activity can be detected. In database environments, defenders must look beyond firewalls and host antivirus. Stored procedures, views, functions, triggers, and permission grants form part of the application's trust boundary. A single unsafe procedure that concatenates user input into dynamic SQL can enable SQL injection even if the front-end application appears to use parameters. Likewise, a procedure marked EXECUTE AS OWNER can allow a low-privilege caller to act with the owner's rights. Extended procedures such as xp_cmdshell can bridge the database engine to the operating system, creating a path from SQL access to host compromise.

1.2 Problem Statement

Many organizations assume that using stored procedures is automatically secure. Industry guidance contradicts that assumption. OWASP notes that stored procedures are safe only when implemented without unsafe dynamic SQL. CIS Benchmarks for SQL Server recommend reducing surface area, for example by disabling xp_cmdshell, and limiting privileges granted to the public role. Without a repeatable audit process, these weaknesses remain invisible until an incident occurs.

1.3 Aim and Objectives

Aim: To audit stored procedures and database objects in a SQL Server laboratory for security weaknesses and to remediate and verify the identified issues using blue-team methods.

- Deploy an isolated SQL Server lab with a sample database suitable for auditing.
- Establish intentionally vulnerable stored procedures that represent realistic weaknesses.
- Inventory database modules and inspect definitions for dangerous patterns.
- Review object permissions and server configuration against CIS-aligned expectations.
- Assess whether SQL Server Audit detective controls are present.
- Document findings with severity ratings and evidence.

- Remediate the findings and re-verify with the same audit scripts.
- Produce a report, repository, and presentation suitable for academic assessment.

1.4 Scope and Limitations

The scope was limited to a local SQL Server 2022 Express instance (localhost\SQLEXPRESS) and the AdventureWorks2022 database. No production systems or unauthorized third-party targets were tested. Limitations include Express edition feature constraints, the use of simulated vulnerable procedures rather than a live enterprise application, and the absence of a full enterprise SIEM correlation stack. These limitations are appropriate for a controlled academic blue-team exercise.

2. Literature and Tooling Review

2.1 CIS Microsoft SQL Server Benchmarks

The Center for Internet Security publishes consensus benchmarks for Microsoft SQL Server. Relevant themes for this project include surface-area reduction, least privilege, and auditing/logging expectations. The project used CIS recommendations as the primary hardening checklist for server configuration and permission review (Center for Internet Security, n.d.).

2.2 OWASP SQL Injection Prevention

The OWASP SQL Injection Prevention Cheat Sheet explains that stored procedures are not inherently immune to injection. Auditors should look for dynamic execution constructs such as EXEC, EXECUTE, and sp_executesql when user input is concatenated into SQL strings. The preferred defenses are parameterized queries and safely implemented procedures (OWASP Foundation, n.d.). Finding F-001 in this project maps directly to that guidance.

2.3 Microsoft SQL Server Audit and Catalog Views

Microsoft documentation describes catalog views such as sys.sql_modules for retrieving module definitions and SQL Server Audit for logging security-relevant events. Audit action groups and database-level execute auditing provide detective controls for dangerous

procedures and configuration changes (Microsoft, n.d.). Finding F-005 addressed the absence of these controls in the initial lab state.

2.4 NetSPI Detective Controls

NetSPI's SQL Server Detective Control Cheat Sheet provides practical patterns for auditing execution of high-risk procedures and for monitoring configuration changes. These patterns informed the design of the project's audit verification and remediation audit specifications (NetSPI, n.d.).

2.5 Tools Used

Tool	Role in the project
SQL Server 2022 Express	Database engine under audit
SQL Server Management Studio 21	Interactive query and administration
AdventureWorks2022	Sample OLTP database
Custom T-SQL audit scripts	Inventory, pattern hunt, permissions, config, audit checks
Windows Application Log	Destination for SQL Server Audit events
Git / GitHub	Version control and submission artifact

3. Methodology

3.1 Laboratory Design

The laboratory was hosted on a local Windows machine. SQL Server Express ran as the named instance SQLEXPRESS. Authentication for administrative auditing used Windows Authentication. AdventureWorks2022 provided realistic schema objects without exposing real personal data.

Figure 1. Laboratory connection summary for the blue-team SQL Server audit.

3.2 Ethical and Safety Controls

All testing remained inside an isolated academic lab. No unauthorized external systems were scanned or exploited. Vulnerable procedures were labeled as lab-only and were not intended for production use.

3.3 Audit Workflow

- Prepare — install engine and tools; restore AdventureWorks2022.
- Seed — deploy intentionally weak procedures to create measurable findings.
- Detect — run inventory and security audit scripts; capture outputs.
- Document — record findings with severity, evidence, and references.
- Remediate and verify — apply fixes; re-run detection scripts; update status.

3.4 Severity Model

Severity	Meaning
Critical	Clear path to data theft via injection or OS command execution
High	Privilege escalation or broadly excessive execute rights
Medium	Missing detective controls that delay incident response
Low	Unnecessary surface area with limited

	immediate impact

4. Implementation

4.1 Environment Preparation

SQL Server 2022 Express and SSMS 21 were installed and verified. AdventureWorks2022 was restored and confirmed online. Project folders were organized as scripts, lab, docs, evidence, report, and presentation.

4.2 Intentionally Vulnerable Procedures

Three procedures were created in AdventureWorks2022 using lab/setup-vulnerable-procs.sql:

- dbo.usp_LabSearchProducts_Unsafe — concatenates @ProductName into dynamic SQL and executes it with EXEC(@sql).
- dbo.usp_LabGetEmployee_Elevate — defined WITH EXECUTE AS OWNER, allowing privilege escalation.
- dbo.usp_LabRunCommand_Dangerous — wraps master.dbo.xp_cmdshell.

All three were granted EXECUTE to public, violating least privilege and simulating a common misconfiguration.

Figure 2. Intentionally vulnerable lab procedures used for detection practice.

4.3 Audit Script Suite

Script	Purpose
scripts/01-inventory.sql	List modules, execute-as mode,

	encryption/visibility
scripts/02-code-patterns.sql	Search definitions for dangerous patterns; list elevated procedures
scripts/03-permissions.sql	Review public grants and execute rights on risky objects
scripts/04-server-config.sql	Check CIS-relevant configuration options and sysadmin membership
scripts/05-audit-check.sql	Inventory server audits and audit specifications

4.4 Remediation Implementation

Remediation was applied with lab/remediate-findings.sql. Unsafe procedures were replaced or removed, public execute rights were revoked, SQL Server Audit was enabled, and remote access was disabled.

Figure 3. Summary of remediation actions applied in the laboratory.

4.5 Configuration Excerpts

Unsafe pattern (pre-remediation):

SET @sql = N'SELECT ... WHERE Name LIKE ''%' + @ProductName + N'%'''; EXEC(@sql);

Safer replacement (post-remediation):

```
SELECT ProductID, Name, ProductNumber FROM Production.Product WHERE Name LIKE N'%' +
@ProductName + N'%';
```

5. Results and Findings

5.1 Summary of Findings

ID	Severity	Category	Object	Status
F-001	Critical	SQL Injection	usp_LabSearchProducts_Unsafe	Fixed
F-002	High	Privilege Escalation	usp_LabGetEmployee_Elevate	Fixed
F-003	Critical	Dangerous Feature	usp_LabRunCommand_Dangerous	Fixed
F-004	High	Excessive Permissions	Lab procs / public	Fixed
F-005	Medium	Missing Detective Controls	SQL Server Audit	Fixed
F-006	Low	Surface Area	remote access	Fixed

Table 1. Findings identified during the stored procedure and database object audit.

5.2 Detection Evidence (Pre-remediation)

Before remediation, script 02-code-patterns.sql returned the unsafe and dangerous procedures and identified the elevated execute-as procedure. Script 03-permissions.sql showed EXECUTE granted to public on all three lab procedures.

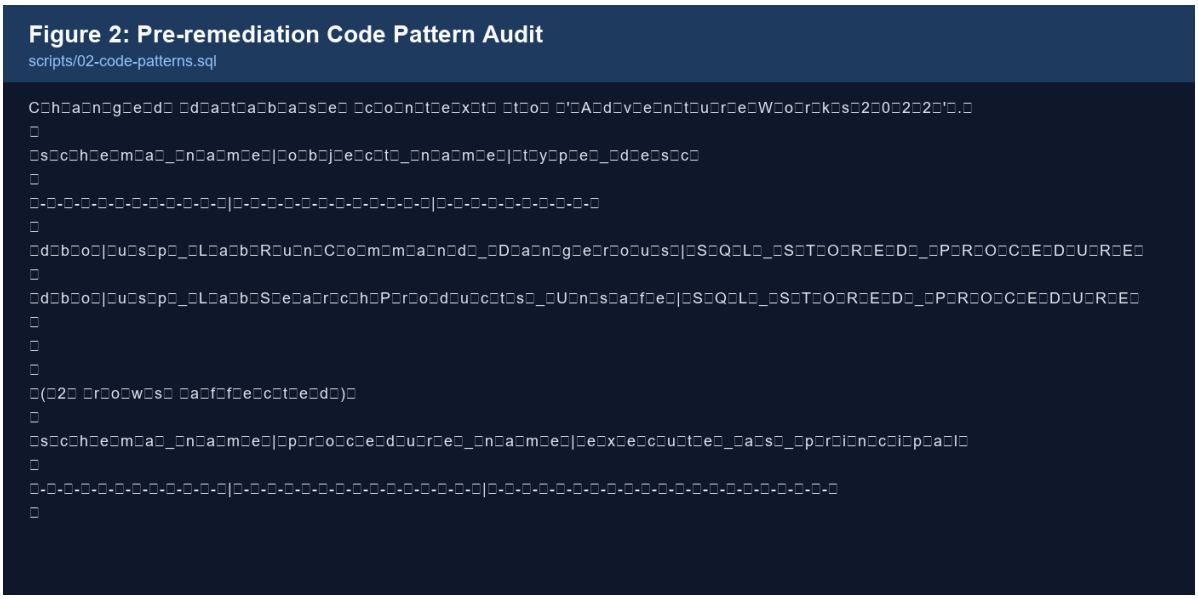

Figure 4. Pre-remediation output of the dangerous code-pattern audit.

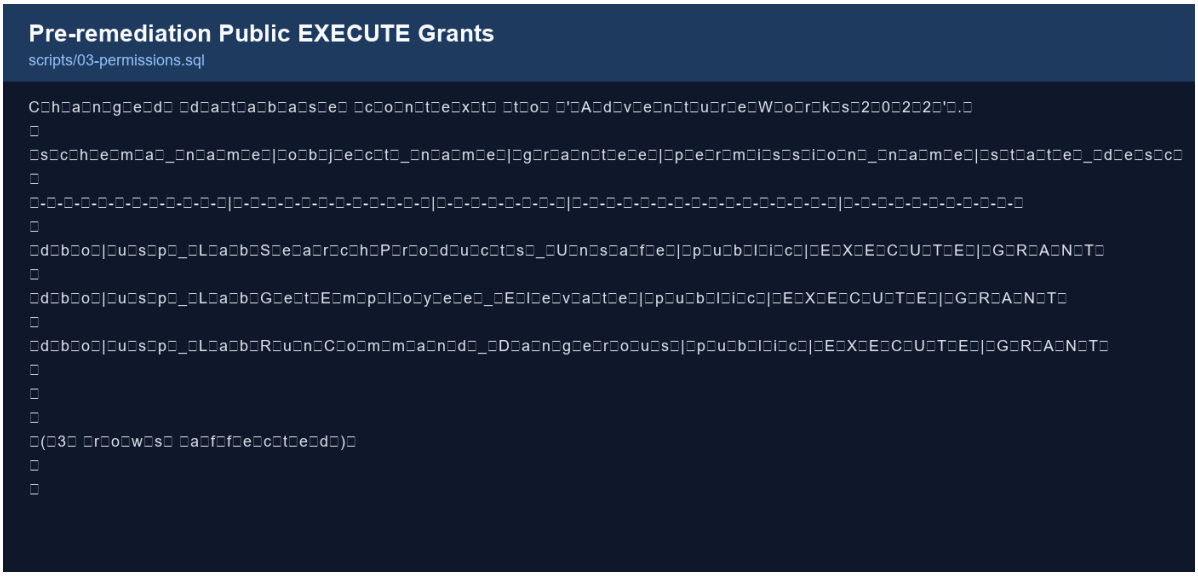

Figure 5. Pre-remediation public EXECUTE grants on lab procedures.

5.3 Finding Narratives

F-001 (Critical) — SQL injection risk.

User-controlled input was concatenated into a dynamic SQL string and executed. An attacker supplying crafted input could alter query logic. This violates OWASP safe stored-procedure guidance.

F-002 (High) — Privilege escalation.

EXECUTE AS OWNER caused the procedure to run with owner rights regardless of the caller's privileges. Combined with public execute rights, this expanded the blast radius of a compromised low-privilege account.

F-003 (Critical) — OS command bridge.

Wrapping xp_cmdshell creates a reusable command-execution interface. Even when xp_cmdshell is disabled at the server level, the presence of such a wrapper is a high-risk code smell and a future enablement hazard.

F-004 (High) — Public execute grants.

Granting execute on sensitive procedures to public means every database principal inherits the right. CIS guidance emphasizes minimizing public permissions.

F-005 (Medium) — No SQL Server Audit.

Without audit specifications, defenders lack reliable telemetry for configuration changes and sensitive procedure execution.

F-006 (Low) — Remote access enabled.

The remote access option increases unnecessary surface area for legacy remote procedure scenarios and was disabled per hardening practice.

5.4 Positive Controls Observed

Even before remediation of the lab procedures, several CIS-aligned controls were already in a good state.

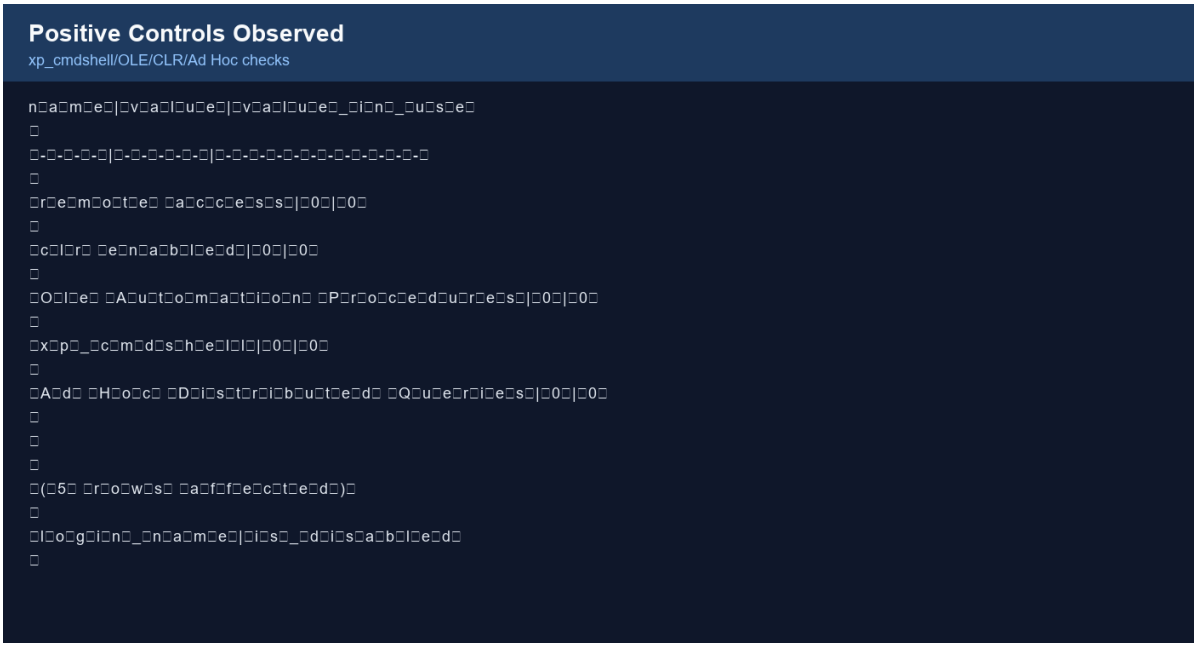

Figure 6. Positive hardening controls already present on the SQL Server instance.

- xp_cmdshell disabled
- OLE Automation Procedures disabled
- CLR disabled
- Ad Hoc Distributed Queries disabled
- sa login disabled

5.5 Post-remediation Verification

After remediation, dangerous code-pattern queries returned zero rows, elevated execute-as procedures were gone, risky public execute grants were removed, BlueTeamLabAudit was started, and remote access showed value = 0 and value_in_use = 0.

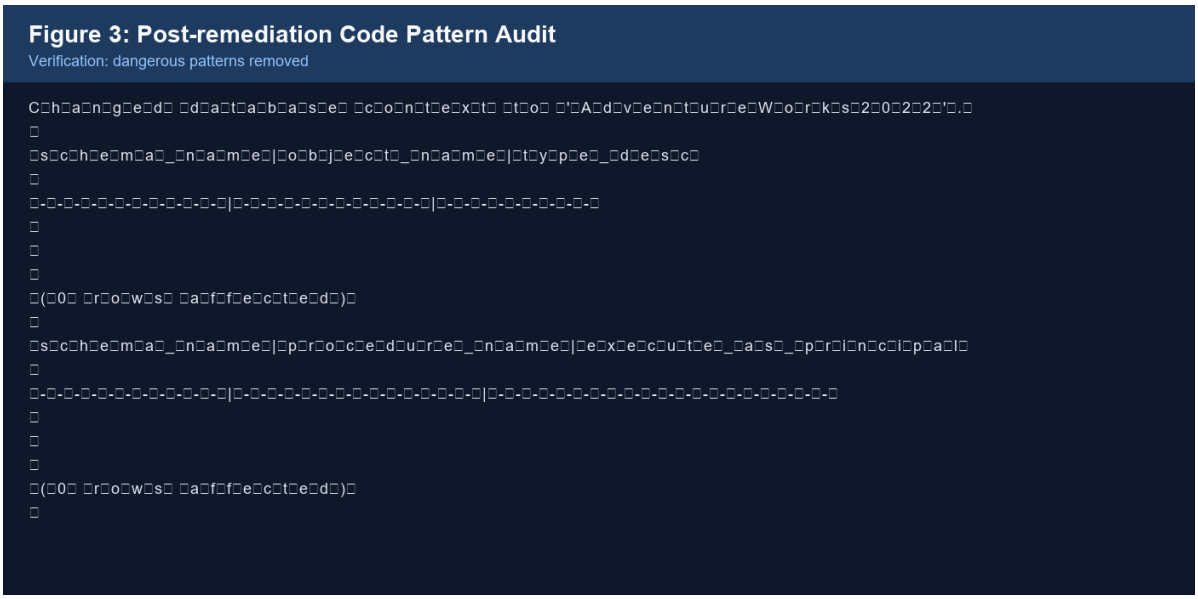

Figure 7. Verification that dangerous stored-procedure patterns were removed.

Figure 8. Verification that risky public EXECUTE grants were removed.

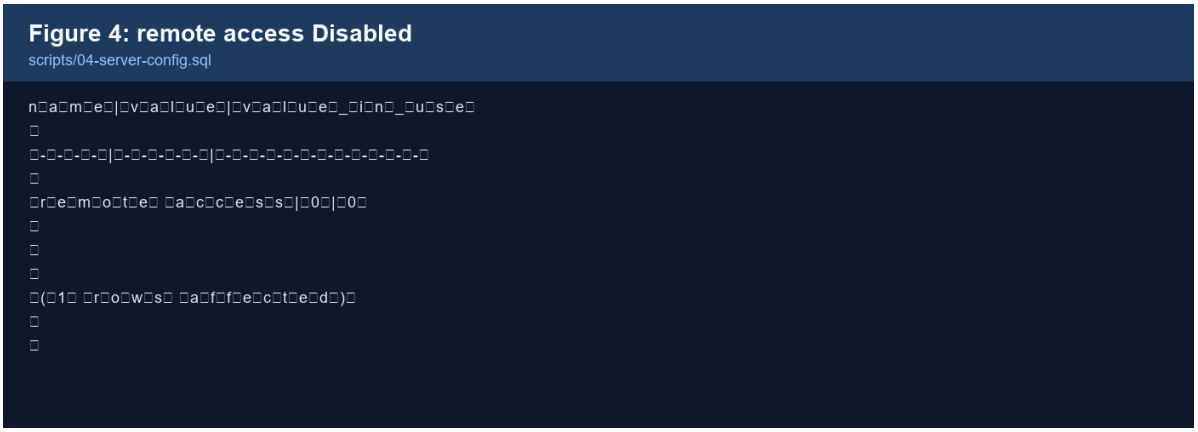

Figure 9. Confirmation that remote access is disabled at runtime.

Figure 10. Detective controls enabled via SQL Server Audit.

6. Analysis and Recommendations

6.1 What the Results Mean

The exercise shows that blue-team database auditing must combine code review, permission review, and configuration review. Focusing on only one layer leaves gaps. For example, disabling xp_cmdshell is necessary but insufficient if application code still contains wrappers and if execute rights are overly broad. Similarly, replacing unsafe procedures without enabling audit leaves defenders blind to future regressions.

6.2 Recommendations for Operational Environments

- Ban unsafe dynamic SQL patterns in code review and automated scanning of sys.sql_modules.
- Avoid EXECUTE AS OWNER unless a documented least-privilege impersonation account is required.

- Keep xp_cmdshell, OLE Automation, and Ad Hoc Distributed Queries disabled unless a formal exception exists.
- Never grant sensitive execute rights to public; use application roles instead.
- Enable SQL Server Audit or equivalent for configuration changes and high-risk object execution; forward events to a SIEM.
- Re-run audit scripts on a schedule and after every schema release.
- Treat remediation as incomplete until verification scripts pass.

6.3 Mapping to Defensive Value

Injection flaws enable data exfiltration; EXECUTE AS OWNER enables privilege escalation; xp_cmdshell wrappers enable host command execution; public execute rights widen abuse; missing audit enables stealthy changes. Fixing each class shrinks attacker opportunity and improves detection and forensics.

7. Conclusion

This Blue Team project delivered a practical audit of stored procedures and database objects on SQL Server 2022 Express using AdventureWorks2022. By seeding realistic weaknesses, applying CIS- and OWASP-informed detection scripts, documenting six findings, remediating them, and verifying the results, the project completed the full defensive lifecycle expected in network monitoring, security, and auditing coursework.

The key lesson is that database security is not only about login passwords. Procedure code quality, permission design, surface-area configuration, and detective auditing together determine whether a database can resist and reveal abuse. The same methodology can be extended to linked servers, SQL Agent jobs, encryption settings, and continuous CIS benchmark compliance scanning.

All findings identified in the laboratory were remediated and verified. The accompanying GitHub repository contains the scripts, documentation, and report materials required for independent review.

8. References

Center for Internet Security. (n.d.). CIS Microsoft SQL Server benchmarks. https://www.cisecurity.org/benchmark/microsoft_sql_server

Microsoft. (n.d.). sys.sql_modules (Transact-SQL). Microsoft Learn. https://learn.microsoft.com/en-us/sql/relational-databases/system-catalog-views/sys-sql-modules-transact-sql

Microsoft. (n.d.). SQL Server audit action groups and actions. Microsoft Learn. https://learn.microsoft.com/en-us/sql/relational-databases/security/auditing/sql-server-audit-action-groups-and-actions

Microsoft. (n.d.). AdventureWorks sample databases. Microsoft Learn. https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure

NetSPI. (n.d.). SQL Server detective control cheat sheet. PowerUpSQL Wiki. https://github.com/NetSPI/PowerUpSQL/wiki/SQL-Server-Detective-Control-Cheat-Sheet

OWASP Foundation. (n.d.). SQL injection prevention cheat sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Portcullis Labs. (n.d.). MS SQL Server audit: Extended stored procedures / table privileges. https://labs.portcullis.co.uk/blog/ms-sql-server-audit-extended-stored-procedures-table-privileges/

9. Appendices

Appendix A — Audit Script List

- scripts/01-inventory.sql
- scripts/02-code-patterns.sql
- scripts/03-permissions.sql

- scripts/04-server-config.sql
- scripts/05-audit-check.sql

Appendix B — Lab Procedure Scripts

- lab/setup-vulnerable-procs.sql
- lab/remediate-findings.sql

Appendix C — Findings Working Sheet

See docs/findings-template.md in the repository for the working findings table used during the audit.

Appendix D — Reproduction Steps

- Install SQL Server 2022 Express and SSMS.
- Restore AdventureWorks2022.
- Run lab/setup-vulnerable-procs.sql.
- Run scripts 01–05 and record findings.
- Run lab/remediate-findings.sql.
- Re-run scripts 02–05 to verify.

Appendix E — Student Declaration

This submission reflects laboratory work completed for CY376 on an isolated SQL Server environment. External standards and tools are cited. No unauthorized systems were tested.

Appendix F — Extended Methodology Notes

The inventory stage used sys.sql_modules joined to sys.objects and sys.schemas to enumerate procedures, functions, triggers, and views. For each object, the audit recorded whether the definition was visible or encrypted and whether an EXECUTE AS clause altered the security context.

The code-pattern stage searched module definitions for EXEC(, EXECUTE(, sp_executesql, xp_cmdshell, sp_OACreate, OPENROWSET, OPENDATASOURCE, OPENQUERY, and EXECUTE

@variable patterns. Matches were manually reviewed to distinguish safe parameterization from unsafe concatenation.

The permission stage inspected sys.database_permissions for grants to public and for EXECUTE rights on objects matching xp_%, sp_OA%, and the project's usp_Lab% procedures. This revealed whether least-privilege principles were being followed.

The configuration stage queried sys.configurations for surface-area options commonly cited in CIS benchmarks. Values and value_in_use were compared because some settings require a service restart before the runtime value changes.

The detective-control stage queried SQL Server Audit DMVs and catalog views to determine whether audits and specifications existed and were enabled. After remediation, BlueTeamLabAudit was confirmed STARTED with server and database specifications active.

Verification repeated the same scripts used for detection. This closed-loop approach prevents the common failure mode where remediation is claimed without measurable evidence that the issue is gone.

Because the environment was Express Edition, some enterprise features were unavailable or limited. Nevertheless, the core blue-team competencies—inventory, detection, documentation, remediation, and verification—were fully exercised and are transferable to Standard and Enterprise deployments.

Future work could integrate the scripts into scheduled Agent jobs, export findings to CSV for ticketing systems, and forward Windows Application log event 33205 into a SIEM for continuous monitoring.